

Trainium/Inferentia2 Architecture Guide for NKI

Print to PDF

Contents

- NeuronCore-v2 Compute Engines
- Data Movement

This document is relevant for: [Trn2](#), [Trn3](#)

In this guide, we will dive into hardware architecture of second-generation NeuronDevices: Trainium/Inferentia2. Our goal is to equip advanced Neuron users with sufficient architectural knowledge to write performant NKI kernels and troubleshoot performance issues on NeuronDevices using [Neuron Explorer](#), a profiler tool designed specifically for NeuronDevices. This guide is also written assuming readers have read through [NKI Language Guide](#) and familiarized themselves with key NKI concepts.

Fig. 44 shows a block diagram of a Trainium and Inferentia2 device. At a high level, both Trainium and Inferentia2 devices consist of:

- 2 NeuronCores (v2).
- 2 HBM stacks with a total device memory capacity of 32GiB and bandwidth of 820 GB/s.
- 32 DMA (Direct Memory Access) engines to move data within and across devices.
- 6 CC-Cores for collective communication.
- 2 (Inferentia2) or 4 (Trainium) NeuronLink-v2 for device-to-device collective communication.

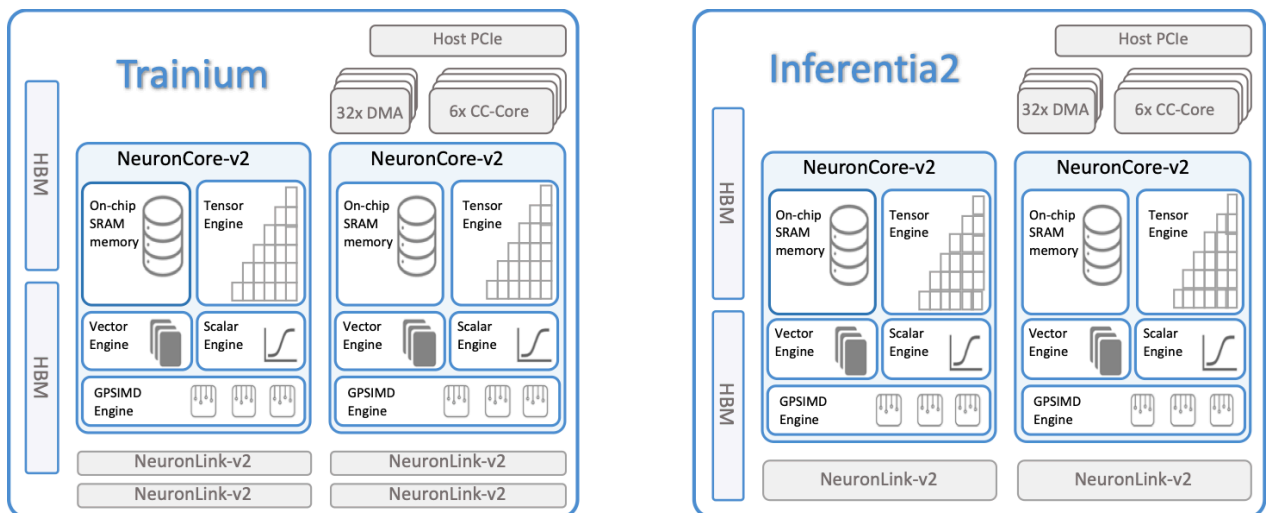


Fig. 44 Trainium/Inferentia2 Device Diagrams.

The rest of this guide will go into details of each compute engine in NeuronCore-v2 and supported data movement patterns across the memory hierarchy.

[latest](#) ▼

[Skip to main content](#)

NeuronCore-v2 Compute Engines

In this section, we will describe the architectural details within a NeuronCore-v2. The figure below is a simplified diagram of the compute engines and their connectivity to the two on-chip SRAMs: state buffer (SBUF) and partial sum buffer (PSUM).

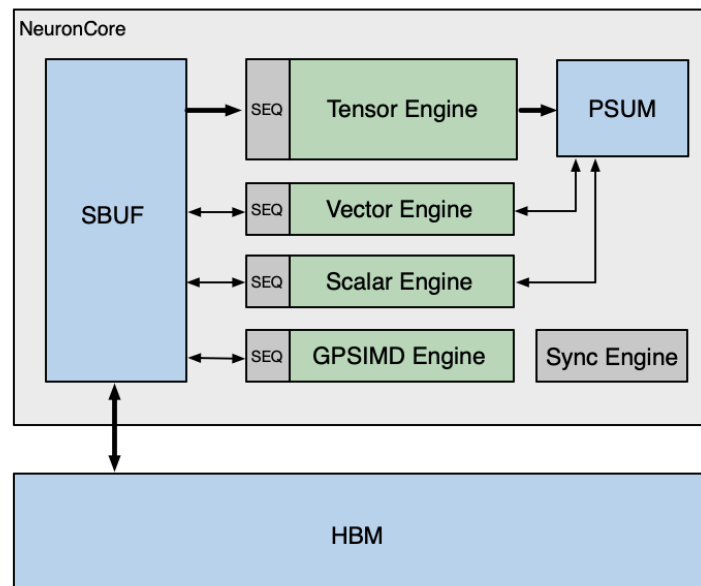


Fig. 45 NeuronCore-v2 and its device memory (HBM).

A NeuronCore-v2 consists of four heterogeneous compute engines (Tensor, Vector, Scalar, and GpSimd), each designed to accelerate different types of operators in modern machine learning models. Each compute engine has its own sequencer, which is responsible for instruction fetch, decode, and issue. The four compute engines execute four independent instruction streams asynchronously in parallel. Explicit synchronization to satisfy data dependencies between engines is handled through atomic semaphores in hardware. In NKI, programmers do not need to program engine synchronization manually. The Neuron Compiler can automatically insert the required synchronizations during compilation, based on data dependencies identified in the NKI kernel.

The instruction stream within each compute engine consists of both control and data-path instructions. Control instructions are executed directly by the engine sequencer and can perform scalar operations using a set of 32-bit scalar registers private to each sequencer. Examples of control instructions include register ALU operations for dynamic condition and address calculations, branching for control flow execution, and triggering DMA transfers. Data path instructions are executed by the specialized engine data path, which interacts with tensors in SBUF/PSUM. Data path instructions can handle flexible addressing and shapes by referencing values stored in scalar registers.

Within each NeuronCore, there is also a Sync Engine, which functions as an engine sequencer that can perform the same types of control instructions. The Sync Engine is most commonly used to trigger DMA transfers without interfering with compute engine instruction scheduling and ordering.

In addition, it is often useful to take engine data-path width and frequency into account when optimizing performance for a multi-engine operator:

Device Architecture	Compute Engine	Data-path Width (elements/cycle)	Frequency (GHz)
Trainium/Inferentia2	Tensor	2x128 (input); 1x128 (output)	2.8
	Vector	128 input/output	1.12
	Scalar		1.4
	GpSimd		1.4

Memory-wise, a NeuronCore-v2 consists of two software-managed on-chip SRAMs, a 24MiB SBUF as the main data storage and a 2MiB PSUM as a dedicated accumulation buffer for Tensor Engine. Both SBUF and PSUM are considered two-dimensional memories with 128 partitions each, i.e., one SBUF partitions has 192KiB of memory while one PSUM partition has 16KiB. We will cover more details on data movements with SBUF/PSUM later [here](#).

The rest of this section will cover the following topics for each compute engine:

- Key functionalities.
- Layout and tile size requirement for input and output tensors.
- Best practices to achieve good performance on the engine.

Tensor Engine

Tensor Engine (TensorE from now on) is specially designed to accelerate matrix-multiplications (matmuls), as well as other operators that can be executed using matrix multiplications such as 2D convolutions. We also note that TensorE can be used for advanced data movement from SBUF to PSUM, including transposition and broadcast (more discussion below [here](#)). Architecturally, the engine is built around a [systolic array](#) with 128 rows and 128 columns of processing elements, which streams input data from SBUF and writes output to PSUM.

Data Types. TensorE supports BF16, FP16, TF32, and cFP8 input matrix data types at a maximum throughput of 92 TFLOPS, as well as 23 TFLOPS for FP32 inputs. TensorE performs mixed-precision calculations, with accumulations at FP32 precision. Therefore, the output data of a TensorE calculation is always in FP32.

Layout. To understand the layout and tiling constraints of TensorE, let’s visualize its connection to SBUF and PSUM as below. Note, PSUM partition dimension is purposely rotated 90 degrees compared to SBUF partition dimension due to systolic array data flow.

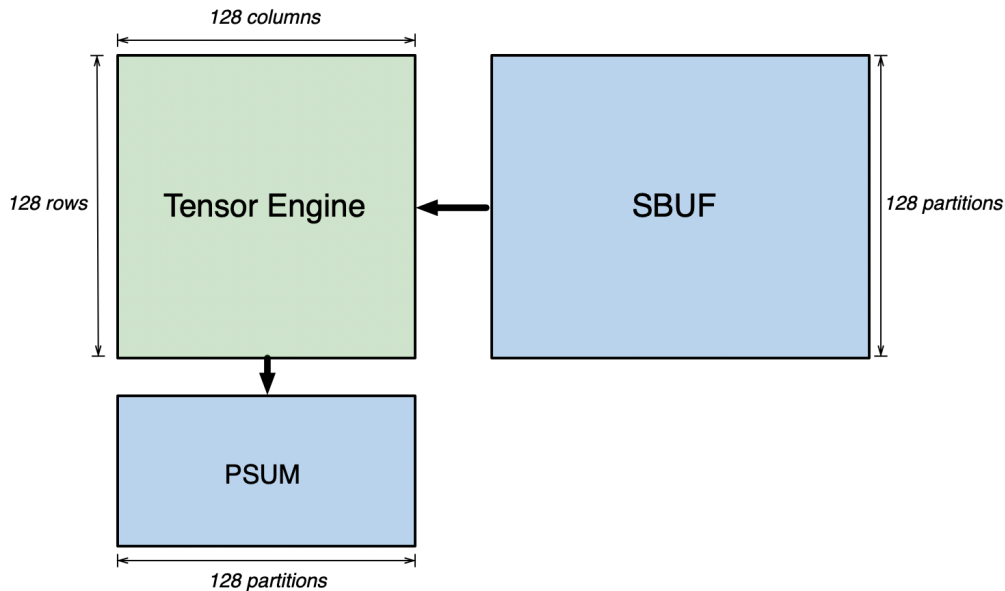


Fig. 46 Tensor Engine and SRAM Connectivity.

As shown in the diagram above, TensorE must **read** input matrices from **SBUF** and **write** output matrices to **PSUM**. PSUM also allows near-memory accumulation of multiple matrix multiplication output tiles (detailed usage discussed [here](#)).

In NKI, to perform a multiplication of two matrices, $x[M, K]$ and $y[K, N]$, you may invoke the NKI ISA API `nki.isa.nc_matmul(x, y)` directly. The returned tile has a shape of $[M, N]$ as expected. At the hardware level, TensorE requires both input tiles to have the **contraction dimension** K in the SBUF partition dimension, that is, the first dimension of input shapes ([Tiling Layout](#)). This ISA requirement is reflected in the low-level API `nki.isa.nc_matmul`, which takes `stationary` and `moving` matrices as input parameters. Therefore, `nki.isa.nc_matmul(x, y)` is a two-step computation: invoking `nki.isa.nc_transpose(x)` to get `stationary` and then `nki.isa.nc_matmul(stationary, moving)` to get the final result. In other words, `nki.isa.nc_matmul(stationary[K,M], moving[K,N])` performs a `stationary.T @ moving` calculation, which will result in an output with dimensions $[M, N]$.

For every `nki.isa.nc_matmul(stationary, moving)` call, TensorE executes two distinct Neuron ISA instructions:

- LoadStationary (short for LS): This instruction loads the `stationary` from SBUF and caches it in internal storage of TensorE
- MultiplyMoving (short for MM): This instruction loads the `moving` from SBUF and multiplies `moving` across the pre-loaded `stationary` matrix from the previous LoadStationary instruction. The output of this instruction is the output of the `nki.isa.nc_matmul` call written to PSUM.

With the above instruction sequence, we as NKI programmers effectively map input tile `stationary` as the stationary tensor and input tile `moving` as the moving tensor for TensorE. As a rule-of-thumb for layout analysis, the **free** axis of the **stationary** tensor always becomes the partition (first) axis of the output tile, while the **free** axis of the **moving** tensor becomes the free axis of the output. [Fig 47](#) below visualizes this concept by showing a matrix multiplication in both mathematical and TensorE views.

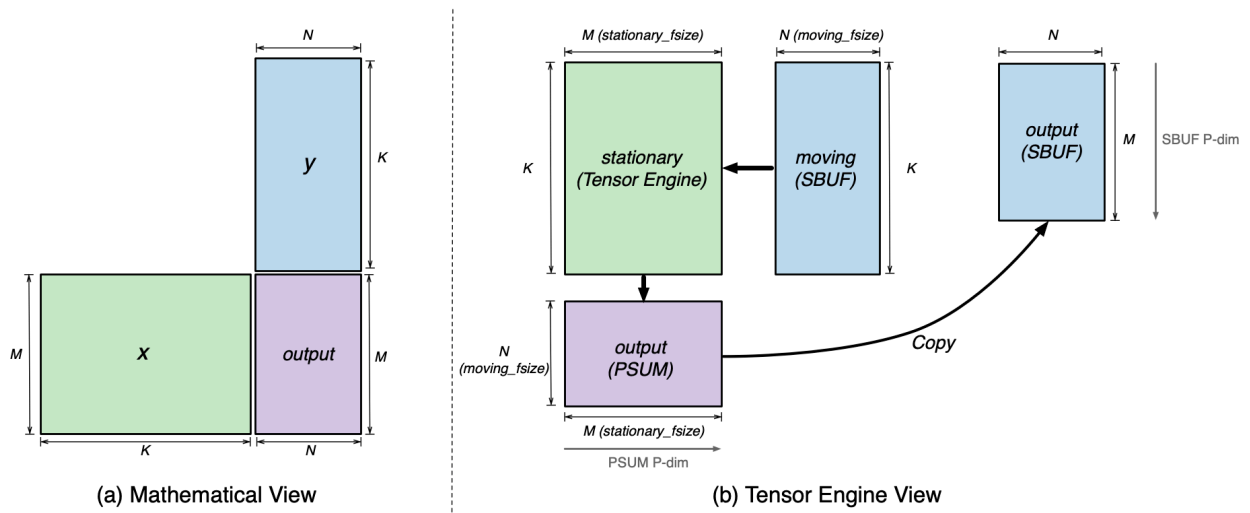


Fig. 47 MxKxN Matrix Multiplication Visualization.

However, programmers are also free to map `stationary` tile to the moving tensor instead, which would lead to the same output tile but transposed: `nki.isa.nc_matmul(moving[K,N], stationary[K,M]) = moving.T @ stationary = outputT[N, M]`. In fact, mapping high-level input tiles to the low-level stationary/moving tensors in TensorE is an important layout decision that NKI programmers should consider to minimize data transposes. Programmers should make this decision based on layout requirements imposed by the compute engine that is going to consume the matrix multiplication output. See NKI Performance Guide for more discussion.

Tile Size. The `nki.isa.nc_matmul` API enforces the following constraints on the input/output tile sizes:

1. `stationary` tensor free axis size (`stationary_fsize`) must never exceed 128, due to the number of PE columns in TensorE.
2. `stationary/moving` tensor partition axis size (`stationary_psize/moving_psize`) must never exceed 128, due to the number of PE rows and also the number of SBUF partitions.
3. `moving` tensor free axis size (`moving_fsize`) must never exceed 512, due to the fact that each `nc_matmul` can only write to a single PSUM bank, which can only hold 512 FP32 elements per PSUM partition.

When the shapes of the input matrices defined in the user-level operator exceed any of the above tile size limitation, we must tile the input matrices and invoke multiple `nki.isa.nc_matmul` calls to perform the matrix multiplication. Exceeding the `stationary_fsize` (#1) or `moving_fsize` (#3) tile limitations for M or N should lead to fully independent `nki.isa.nc_matmul` with disjoint output tiles. However, when `K` exceeds the `stationary_psize/moving_psize` limit, we need to tile the input matrices in the contraction dimension and invoke multiple `nki.isa.nc_matmul` to accumulate into the *same* output buffer in PSUM. Refer to the [Tiling Matrix Multiplications](#) tutorial for a NKI code example.

Alternative Use Case

One interesting use case of TensorE is low-latency data reshape within NeuronCore, which typically involves multiplying a matrix to be reshaped with a compile-time constant matrix filled with zeros and ones.

As an example, we can perform a 128x128 matrix transposition (i.e., swap the free and partition axis of the matrix) using `nki.isa.nc_matmul(transpose_input, identity)`, where `transpose_input` is the matrix to be transposed and `identity` is a 128x128 identity matrix. In fact, this is exactly what `nki.isa.nc_transpose()` does. [latest](#)

[Skip to main content](#)

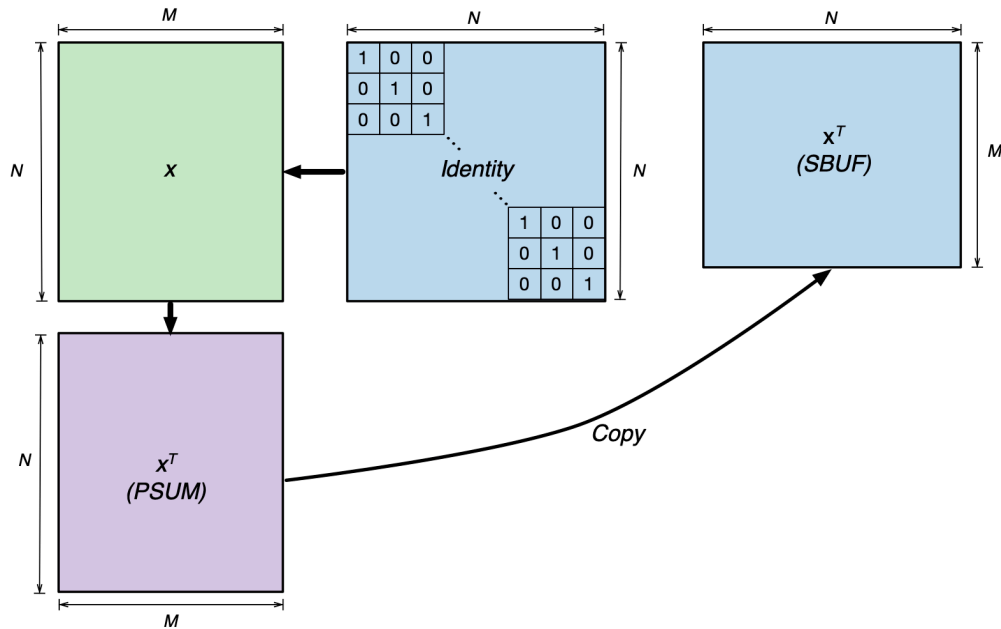


Fig. 48 Transposition.

Similarly, we can broadcast a vector occupying a single partition to M ($M \leq 128$) partitions using `nki.isa.nc_matmul(ones, broadcast_input, is_stationary_onezero=True)`, where `ones` is a $1 \times M$ vector filled with ones and `broadcast_input` is the vector to be broadcast. In fact, NKI invokes such `matmul` under the hood when `broadcast_input.broadcast_to((M, broadcast_input.shape[1]))` is called.

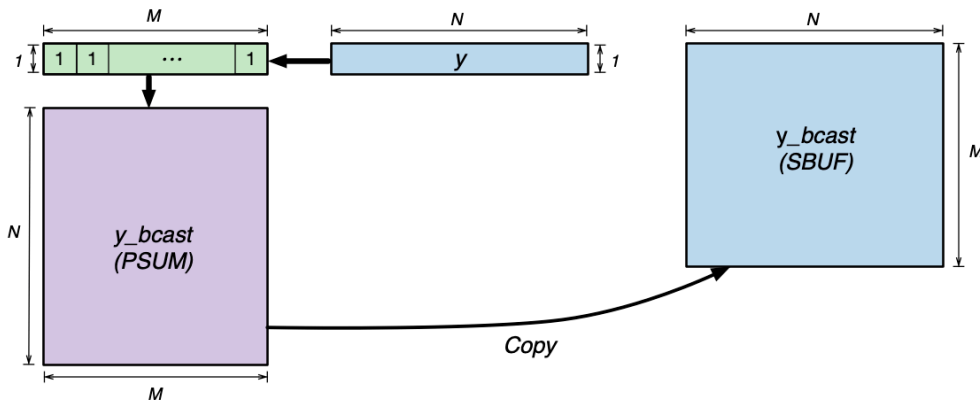


Fig. 49 Partition Broadcast.

In general, we can achieve many more complex data reshapes in TensorE, such as shuffling partitions of a SBUF tensor, by constructing appropriate zero/one patterns as one of the `matmul` inputs.

Finally, we can also leverage TensorE for data summation across SBUF partitions (P-dim summation). For example, a vector laid out across SBUF partitions can be reduced into a single sum using TensorE as shown in the diagram below. Note, this utilizes only a single PE column of the TensorE; therefore, depending on the surrounding operators, this may not be the best use of TensorE. If you can do summation within each partition (F-dim summation), see [nki.isa.tensor_reduce](#) for an alternative reduction implementation on Vector Engine. It is recommended to choose the engine based on the natural layout of your input data to avoid any transpositions.

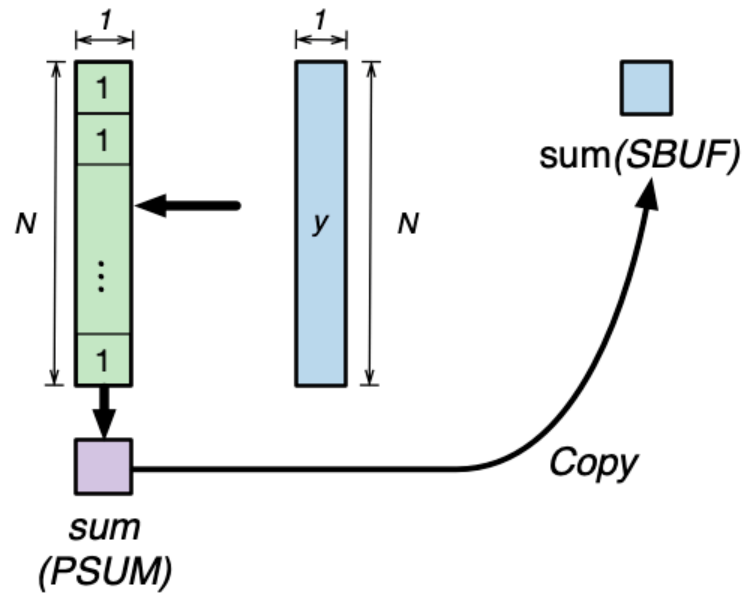


Fig. 50 Cross-Partition Accumulation

As TensorE is the most performant compute engine of the NeuronCore in terms of FLOPS, the goal is to have it execute meaningful computation at high utilization as much as possible. The above “alternative use cases” stop TensorE from performing *useful* computations at *high* throughput and therefore, should generally be avoided. However, there are situations where it is advisable to use them:

- Operators that do not require heavy matmuls anyhow, e.g. normalization, softmax.
- Layout conflicts between producer and consumer engines where broadcast/transpose are absolutely unavoidable (see example in fused attention tutorial).

Performance Consideration

As a rule of thumb, TensorE can achieve the best throughput when it runs many back-to-back

`nki.isa.nc_matmul` with both input matrices at the largest possible tiles sizes (`stationary` is 128x128 and `moving` is 128x512). In this ideal scenario, TensorE sees the below instruction sequence:

- `LoadStationary (LS[0])` (128x128)
- `MultiplyMoving (MM[0])` (128x512)
- `LoadStationary (LS[1])` (128x128)
- `MultiplyMoving (MM[1])` (128x512)
- ...

Cost Model: TensorE is a deeply pipelined engine; therefore, the engine can have several `LS&MM` instruction pairs in-flight at a given time. Due to this pipelining nature, it is often *not* useful to use end-to-end execution *latency* of a single instruction when estimating the instruction cost. Instead, we can focus on the **initiation interval** of such instructions, that is, the number of cycles between successive instruction launches. Therefore, we can estimate the cost of an instruction `I` by how soon TensorE can issue the next instruction after `I`.

For the sake of discussion, let's assume we have many back-to-back `MM` instructions with BF16/FP16/TF32/cFP8 input data type that reuse a single pre-loaded `stationary` inside TensorE. The initiation interval between subsequent MM instructions in this case is roughly `max(N, MM_INIT_LATENCY)`, where `MM_INIT`

FP32 input data type, the instruction cost is roughly 4x higher than BF16/FP16/TF32/cFP8. Therefore, whenever possible, we recommend down-casting FP32 input matrix data type to one of BF16/FP16/TF32/cFP8 before performing matrix multiplications.

Figure below visualizes two pipelined `MM` instructions:

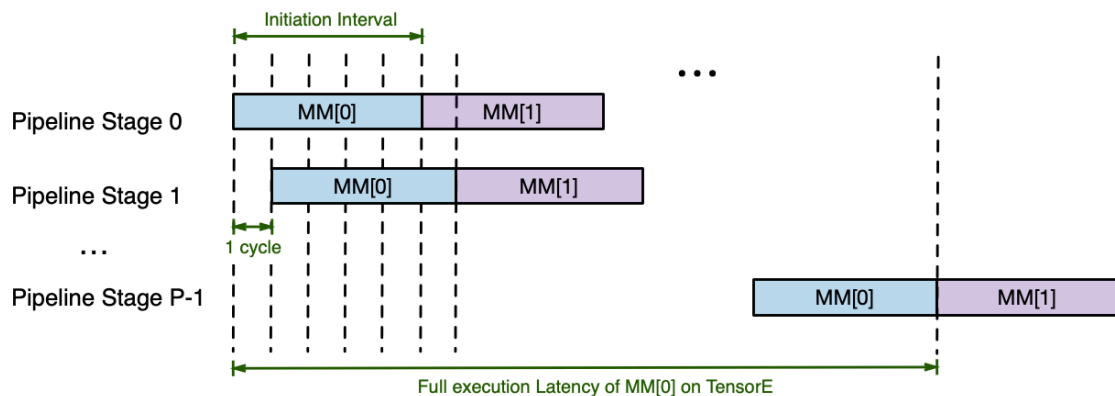
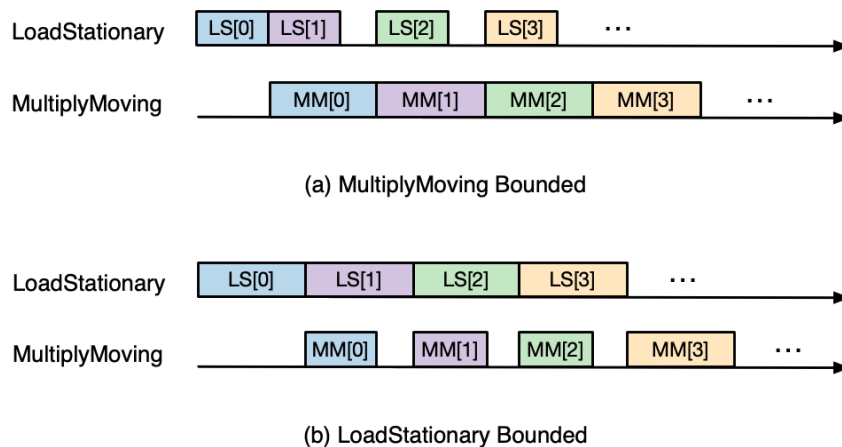


Fig. 51 Pipelined multiplyMoving instructions.

Background LoadStationary: In typical workloads, TensorE would be alternating between LS and MM instructions with different input matrices. In order to optimize TensorE's utilization, we also enable a "background LoadStationary" capability, which allows loading of the next stationary tensor in parallel to the computation on the current stationary tensor.

As a result, depending on the relative sizes of the `stationary` and `moving` matrices, the overall TensorE performance can be bounded by either `LS` or `MM` instructions. Figure below visualizes these two cases. In the ideal scenario where `stationary` and `moving` use the largest tile sizes, TensorE should operate in case (a).



Possible execution timeline execution with background LoadStationary

Fast LoadStationary: Since `LoadStationary` is a pure data movement with no computation, TensorE can perform `LoadStationary` up to 4x faster than a `MultiplyMoving` with the same free axis size. Fast `LoadStationary` has an important performance implication on `nki.isa.nc_matmul`: When one of the input matrices has a small free axis size and the other has a large free axis size, we prefer to put the matrix with large free axis as the `stationary` matrix. For example, if we try to do a vector-matrix multiplication, it is recommended to put the matrix as `stationary` matrix and vector as `moving` matrix to get the best performance out of TensorE.

[latest](#) ▼

[Skip to main content](#)

Vector Engine

Vector Engine (VectorE) is specially designed to accelerate vector operations where every element in the output tensor typically depends on multiple elements from input tensor(s), such as vector reduction and element-wise operators between two tensors. VectorE consists of 128 parallel vector lanes, each of which can stream data from a SBUF/PSUM partition, perform mathematical operations, and write data back to each SBUF/PSUM partition in a deeply pipelined fashion.

Data Types. VectorE supports all NKI data types (details see [supported data types in NKI](#)) in both input and output tiles. [Arithmetic operations](#) are performed in FP32, with automatic zero-overhead input and output casting to and from FP32. Refer to `nki.isa` API reference manual for any instruction-specific data type requirements.

Layout & Tile Size. VectorE instructions expect the parallel axis of the input and output data to be mapped to the partition dimension. For example, the figure below shows reduction add of a NxM matrix along the M dimension. Since each of N rows in the matrix can be reduced in parallel, the N dimension of the matrix should be mapped to the SBUF partition dimension. Refer to the [nki.isa API manual](#) for instruction-specific layout constraint of different VectorE instructions.

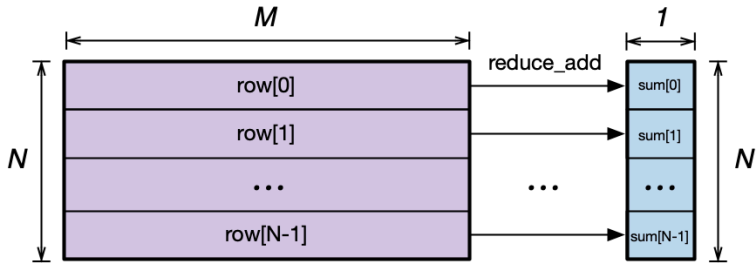


Fig. 52 Reduce add on Vector Engine.

In terms of tile size, the majority of VectorE instructions only have limitation on the input/output tile partition dimension size which must not exceed 128, while the free dimension size can be up to 64K elements for SBUF or 4K elements for PSUM. However, there are a few notable exceptions, such as `nki.isa.bn_stats` which further imposes free dimension size of input tile cannot exceed 512. Refer to the *nki.isa API manual* `<nki.language>` for instruction-specific tile size constraints.

Cross-partition Data Movement

The VectorE also supports a limited set of cross-partition data movement within each group of 32 partitions. The figure below shows connectivity between SBUF and VectorE banks. VectorE consists of four Reshape and Compute banks: each Reshape Bank connects to 32 SBUF/PSUM partitions and outputs 32 parallel streams of data, while each Compute Bank can process 32 parallel data streams using 32 vector lanes. The Compute Bank can write back to 32 SBUF/PSUM partitions.

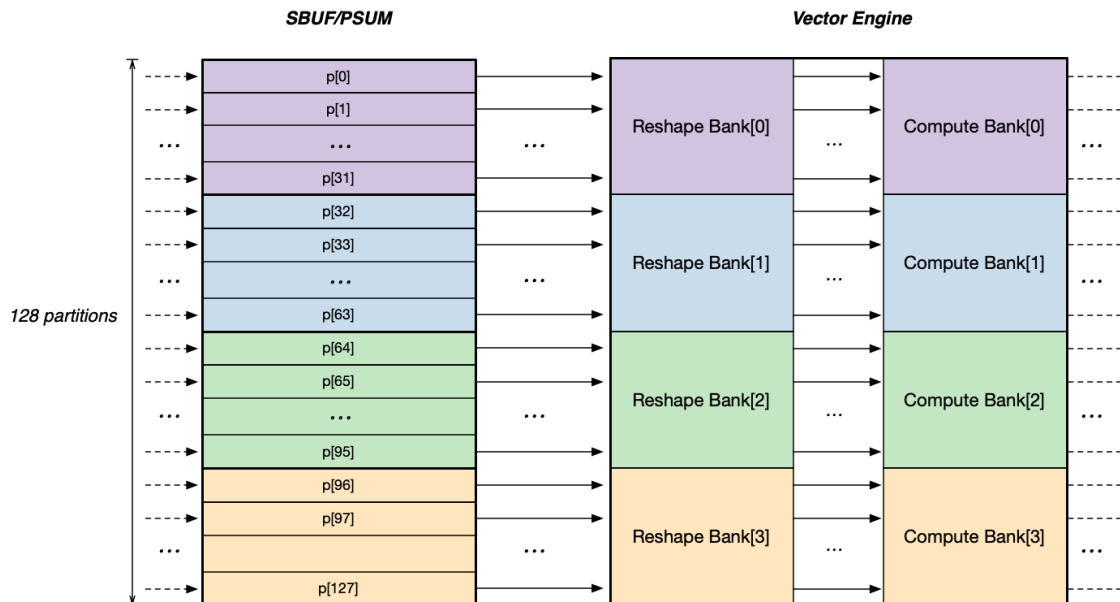


Fig. 53 Vector Engine reshape and compute banks.

The Reshape Bank supports the following data movement:

1. *32x32 transpose*: Each Reshape Bank can read in 32 elements per SBUF/PSUM partitions and transpose the partition and free dimension of the incoming 32x32 matrix. This can be invoked by `nki.isa.nc_transpose` API by selecting VectorE as the execution engine.
2. *32 partition shuffle*: Each Reshape Bank can take an arbitrary *shuffle mask* `SM*` of length 32. The integer value of `SM[i]` indicates the source partition ID (modulo 32) that the Reshape Bank output stream `i` will get. For example, we can broadcast partition[0] to partition[0-31] using a SM of 32 zeros. This can be invoked by `nki.isa.nc_stream_shuffle` API.

Refer [here](#) later in this doc for cross-bank data movement.

Performance Consideration

128 Parallel Compute Lanes: VectorE can perform computation with all 128 vector lanes in parallel, with each lane streaming data from/to one SBUF/PSUM partition. Therefore, the performance cost of a VectorE instruction using all 128 lanes is the same as an instruction that uses fewer than 128 lanes.

As a result, we recommend NKI developers to maximize the compute lanes used per VectorE instruction, that is, the partition axis size of input/output tiles of a single `nki.isa` or `nki.language` compute API call. When the partition axis size of input tiles is inevitably fewer than 128 partitions due to high-level operator definition, we could adopt an optimization called “partition vectorization” by packing multiple “small” VectorE instructions of the same operation into a single “large” Vector instruction. Refer to NKI Performance Guide for more detailed discussion of this optimization.

Cost Model: In the most common cases where the free axis size (`N`) of the input tile(s) is sufficiently large (`N > 128`), the execution cost of an instruction on VectorE is correlated to `N`:

- If there is only one input tile, most VectorE instructions can execute in roughly `N` cycles (example: `nki.isa.tensor_scalar`)
- If there are two input tiles, the instruction can execute in roughly `2N` cycles (example: `nki.isa.tensor_tensor`),

[latest](#) ▼

[Skip to main content](#)

There are a few exceptions to the above rule, depending on the data types and instruction type. See [NKI ISA API doc](#) for instruction-specific instruction cost details.

In the rare cases where VectorE is running many back-to-back instructions either with `N << 128` or with every instruction depending on the output tile of the previous instruction, we need to add a static instruction overhead of 100 engine cycles to the above execution cost estimate.

The above rules are for general guidance only. To find out the exact instruction costs for your NKI kernel, you may capture a detailed instruction execution trace on device using [neuron-profiler](#).

Scalar Engine

Scalar Engine (ScalarE) is specially designed to accelerate scalar operations where every element in the output tensor only depends on one element of the input tensor. In addition, ScalarE provides hardware acceleration to evaluate non-linear functions such as Gelu and Sqrt. The currently supported set of non-linear functions is listed in [here](#). It is worth noting that we can support any new non-linear functions on ScalarE as they come up in new ML model architectures through Neuron SDK software updates. Similar to VectorE, ScalarE consists of 128 parallel lanes, each of which can stream data from a SBUF/PSUM partition, perform mathematical operations, and write data back to each SBUF/PSUM partition in a deeply pipelined fashion.

Data Types. ScalarE supports all NKI data types (details see [supported data types in NKI](#)) in both input and output tiles. All internal computation is performed in FP32, with automatic zero-overhead input and output casting to and from FP32.

Layout & Tile Size. ScalarE typically evaluates scalar operations (such as `nki.language.gelu`), which does not impose any input/output tile layout constraints. However, there are additional hardware features in ScalarE that will have layout constraints similar to VectorE (more discussion later).

In terms of tile size, ScalarE instructions only have limitation on the input/output tile partition dimension size which must not exceed 128, while the free dimension size can be up to 64K elements for SBUF or 4K elements for PSUM.

Pipelined Multiply-Add

Each ScalarE compute lane also supports an additional multiply-add **before** the non-linear function (`func`) is applied in a pipeline fashion. Mathematically, ScalarE implements:

```
# Case 1: scale is SBUF/PSUM vector
# Input: 2D in_tile, 1D scale, 1D bias
# Output: 2D out_tile
for lane_id in range(in_tile.shape[0]):
    for k in range(in_tile.shape[1]):
        out_tile[lane_id][k] = func(in_tile[lane_id][k] * scale[lane_id]
                                   + bias[lane_id])

# Case 2: scale is a compile-time scalar constant in the instruction
for lane_id in range(in_tile.shape[0]):
    for k in range(in_tile.shape[1]):
        out_tile[lane_id][k] = func(in_tile[lane_id][k] * scale
                                   + bias[lane_id])
```

This functionality can be invoked using the `nki.isa.activation` API by specifying a `scale` for multiply and `bias` for addition. The scale can either be a tile from SBUF/PSUM with one element/partition or a compile-time constant.

[Skip to main content](#)

mental model for this capability is combining a `nki.isa.tensor_scalar` instruction with a non-linear function evaluation into a single instruction (2x speed-up than two separate instructions).

Pipelined Reduction

Each ScalarE compute lane also supports reduction **after** the non-linear function (`func`) is applied in a pipeline fashion. On NeuronCore-v2, the reduction operator can only be addition.

Mathematically, ScalarE with accumulation enabled implements:

```
# Input: 2D in_tile, 1D scale (similarly for scalar scale), 1D bias
# Output: 2D out_tile, 1D reduce_res
for lane_id in range(in_tile.shape[0]):
    for k in range(in_tile.shape[1]):
        out_tile[lane_id][k] = func(in_tile[lane_id][k] * scale[lane_id]
                                   + bias[lane_id])
        reduce_res[lane_id] += out_tile[lane_id][k]
```

This functionality can be invoked using the `nki.isa.activation_reduce` API by specifying `reduce_op` as `nki.language.add` and `reduce_res` as the output reduction tile, passed by reference.

A useful mental model for this capability is combining a `nki.isa.activation` instruction with a `nki.isa.tensor_reduce` into a single API, which returns results from **both** APIs. Note, `nki.isa.activation_reduce` invokes two back-to-back ISA instructions on hardware, *Activate* and *ActReadAccumulator*. The *Activate* instruction performs the regular computation as specified in `nki.isa.activation` and also reduction at no additional cost. The reduction result is cached inside ScalarE after *Activate*. The *ActReadAccumulator* instruction is a low cost (roughly 64 ScalarE cycles on NeuronCore-v2) instruction to write the internal reduction result back to SBUF/PSUM, one element per partition.

Performance Consideration

All the performance notes discussed for [Vector Engine](#) earlier are applicable to Scalar Engine, with one exception regarding instruction cost for two input tensors - ScalarE can only read up to one input tensor per instruction.

Instruction Combination. All `nki.isa.activation` instructions have the same execution cost, regardless of whether we enable the scale multiplication or bias add. Therefore, it is recommended to combine such multiply-add operations with non-linear function evaluation into a single ScalarE instruction if the computation allows it. This is highly useful for ML operators that are **not** TensorE heavy (not matmul-bound). Softmax is one such example, where we typically subtract the maximum value of the input elements before evaluating exponential function for numerical stability.

GpSimd Engine

GpSimd Engine (GpSimdE) is intended to be a general-purpose engine that can run any ML operators that cannot be lowered onto the other highly specialized compute engines discussed above efficiently, such as applying a triangular mask to a tensor.

A GpSimdE consists of eight fully programmable processors that can execute arbitrary C/C++ programs.

Therefore, this engine provides the hardware support for [Neuron Custom Operator](#). In addition, each processor is a 512-bit vector machine that can run high-performance vectorized kernels. Every `nki.isa` AI  [latest](#) ▼

GpSimdE such as [this](#) [page](#) [document](#) [implementation](#) that [Neuron](#) [engine](#) [has](#) [done](#) [for](#) [this](#)

[Skip to main content](#)

Data Types. Each processor in GpSimd supports vectorized computation for

- 16x FP32/INT32/UINT32, or
- 32x FP16/INT16/UINT16, or
- 64x INT8/UINT8

This is in contrast to ScalarE/VectorE which can only perform arithmetic operations in FP32. However, if the GpSimdE program chooses to, it can also access SBUF data of any supported data types in NKI and perform data casting to- and from-FP32 at no throughput cost similar to VectorE/ScalarE.

Layout & Tile Size. The layout and tile size requirements of GpSimdE highly depend on semantics of the exact instruction. Refer to the [nki.isa API reference guide](#) for these requirements.

Memory Hierarchy. In Trainium/Inferentia2, each GpSimdE processor has 64KB of local data RAM, also called tightly-coupled memory (TCM) as discussed in [Neuron Custom Operator](#). The TCM is configured with a 3-cycle access latency and 512-bit data width. Therefore, TCM is often used to store intermediate computation results within a Neuron Custom Operator or GpSimdE instruction.

The eight processors in GpSimdE also have a high-bandwidth read/write interface connected to the SBUF. [Figure 54](#) below illustrates the GpSimdE connectivity to SBUF. Each processor connects to 16 SBUF partitions for both reading and writing: processor[0] connected to partition[0:15], processor[1] to partition[16:31] and so on. Each processor can programmatically send tensor read/write requests to SBUF to access data from the connected partitions. On the read side, once a read request is processed, the tensor read interface can deliver up to 512-bit of data from all 16 connected partitions collectively (up to 32-bit per partition) to the processor per cycle, which matches the 512-bit SIMD width. Similarly, on the write side, the tensor write interface can accept 512-bit of data for writing back to the connected SBUF partitions per cycle.

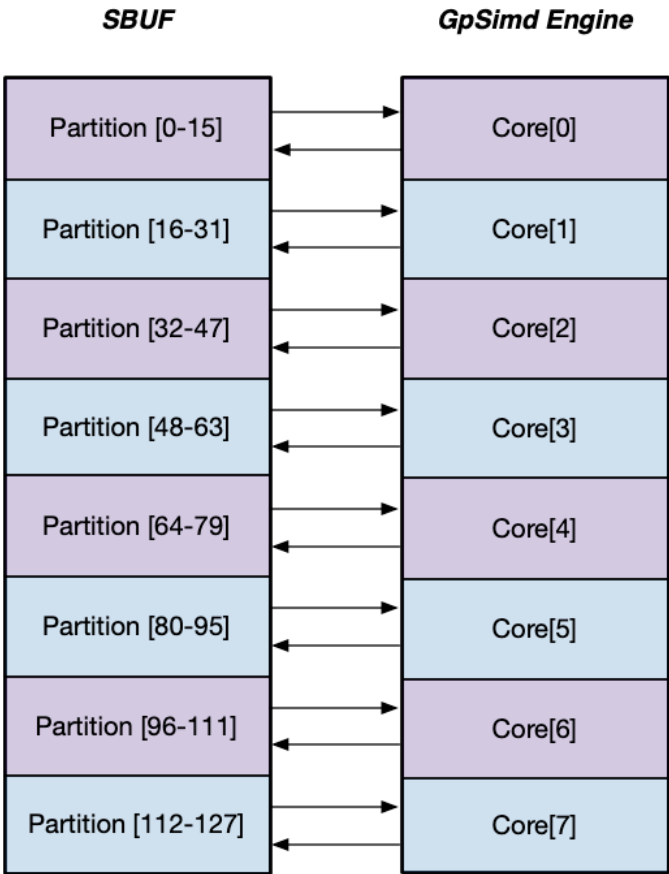


Fig. 54 Connectivity between GpSimdE and SBUF

[Skip to main content](#)

Performance Consideration

128 Parallel Compute Lanes: Similar to VectorE and ScalarE, GpSimdE has 128 parallel compute lanes for 32-bit computation data types across SIMD lanes of all eight processors. Therefore, it is desirable to invoke GpSimdE instructions that will utilize all the parallel compute lanes, typically through accessing all 128 SBUF partitions for input and output. In addition, since each processor can also handle 32-wide 16-bit or 64-wide 8-bit data type computation, GpSimdE can effectively support 256 or 512 parallel compute lanes internally.

Cost Model: Unlike VectorE/ScalarE, there is no rule-of-thumb to estimate execution cost of a GpSimdE instruction. Refer to the [nki.isa](#) API reference manual to find out instruction-specific latency estimates.

Data Movement

In this section, we will dive into the memory subsystem and discuss how to perform data movement between different memories and also how to do it efficiently. As a reminder, there are three main types of memory on a NeuronDevice: HBM, SBUF, and PSUM, from highest to lowest capacity. Figure below shows the specifications of these memories and their connectivity for one NeuronCore-v2:

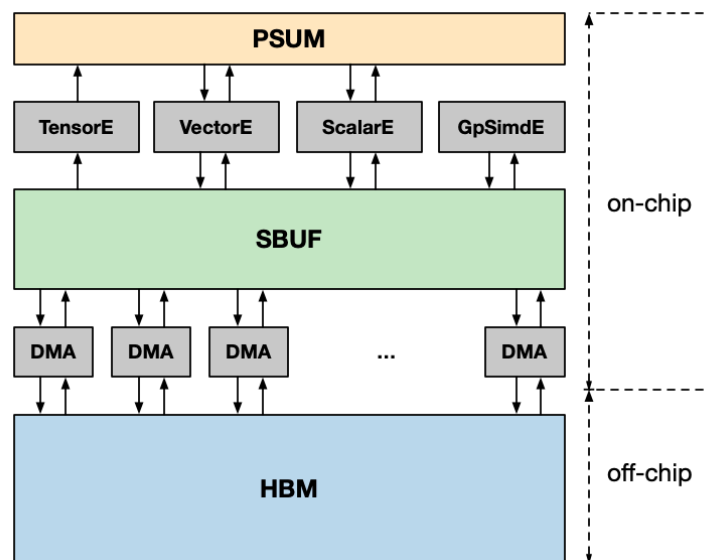


Fig. 55 Memory hierarchy.

As shown in the above figure, data movement between HBM and SBUF is performed using on-chip DMA (Direct Memory Access) engines, which can run in parallel to computation within the NeuronCore. Data movement between PSUM and SBUF is done through ISA instructions on the compute engines. However, different compute engines have different connectivity to SBUF/PSUM as indicated by the arrows in the figure. In addition, NeuronCore-v2 has the following restrictions:

1. VectorE and GpSimdE cannot access SBUF in parallel.
2. VectorE and ScalarE cannot access PSUM in parallel.

Therefore, VectorE and GpSimdE instructions that access SBUF must be serialized, similarly for VectorE and ScalarE instructions that access PSUM. This is enforced by Neuron Compiler during NKI kernel compilation, so NKI developers are not required to program such serializations.

The rest of this section will discuss the following topics in detail:

[latest](#) ▼

[Skip to main content](#)

- Accessing SBUF/PSUM tensors using compute engines.
- In-memory accumulation using TensorE and PSUM.

Data movement between HBM and SBUF using DMAs

Each NeuronCore-v2 is equipped by 16 parallel DMA engines that can perform data movement between any addressable memories in the system. Here, we focus on using these DMA engines to move data between the local SBUF and HBM. Each DMA engine can process one **DMA transfer** at a time driving a peak bandwidth of 27 GiB/s, but all DMA engines can process different DMA transfers in parallel.

Each DMA transfer can gather a list of source **DMA buffers** and then scatter the data into another list of destination DMA buffers. Data within a DMA buffer must be continuous in the memory address map. There is some performance overhead at both DMA buffer and transfer levels, both of which can be amortized by moving a sufficiently large amount of data (more discussion below).

Next, let's examine how HBM and SBUF are laid out in the device memory address map. On one hand, HBM is logically a one-dimensional memory and hence occupies a flat chunk of continuous addresses in the address map. In the most common cases, an HBM tensor in NKI is also contiguous in the HBM address space.

On the other hand, SBUF is considered a two-dimensional memory with 128 partitions as discussed earlier [here](#). [Figure 56](#) shows how SBUF addresses fit in the device address map. `sbuf_base_addr` is a 64-bit address dependent on which NeuronCore-v2 on the device the SBUF is located in. The SBUF addresses start from the first byte of partition 0, increment along the free dimension first and then advance onto the next partition.

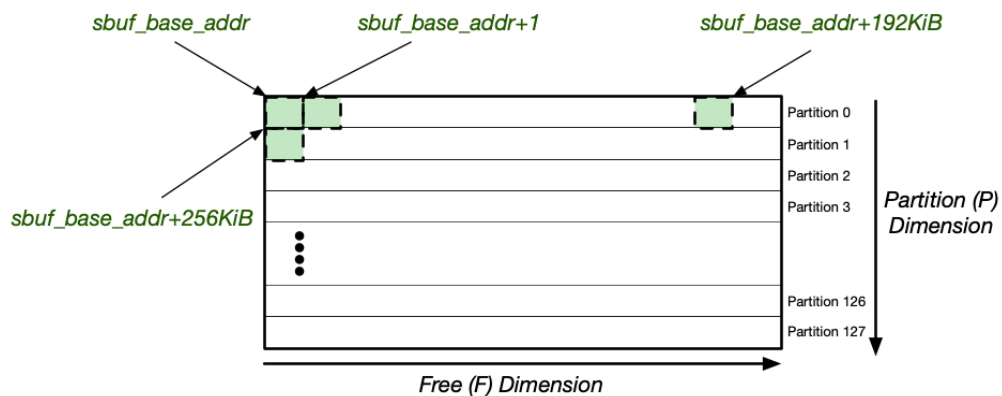
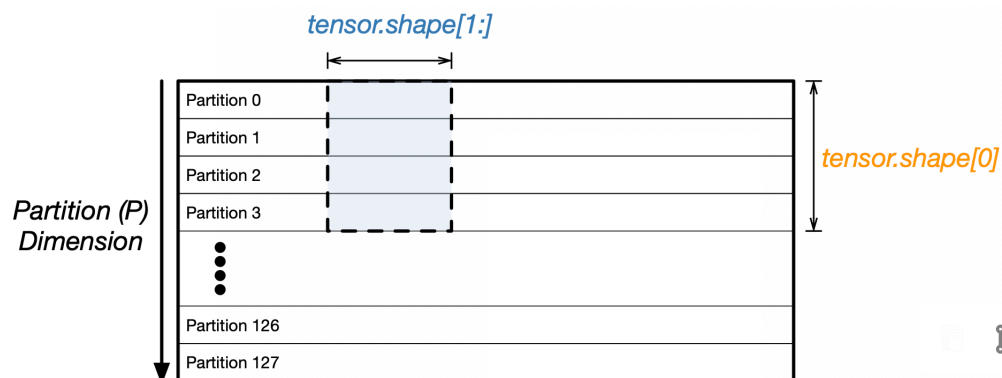


Fig. 56 SBUF memory address space.

As discussed in [NKI Language Guide](#), an SBUF tensor in NKI spans one or more partitions, with data starting at the same offset:



[Skip to main content](#)

Fig. 57 SBUF tensor.

As a result, a data movement involving `tensor` in SBUF will require at least `tensor.shape[0]`, i.e., P dim size, different DMA buffers, since slices of tensor data from different SBUF partitions occupy non-contiguous memory in the address space. If the tensor data slice within each SBUF partition is not contiguous in the F dimension, more DMA buffers will need to be unrolled along the F dim. These DMA buffers are typically grouped into different DMA transfers so that multiple DMA engines can participate in the data movement to maximize memory bandwidth utilization.

In NKI, moving data from HBM to SBUF and from SBUF to HBM are done with calls to the `nki.isa.dma_copy` API. Neuron Compiler is responsible for converting each NKI API call to DMA transfers and assigning these transfers to different DMA engines. As an example, loading a 128x512 FP32 HBM tensor to SBUF is best done through 16 DMA transfers (one per DMA engine), each moving a scatter-gather list of 8 DMA buffers:

```
import nki.language as nl
tile = nl.load(in_tensor[0:128, 0:512])
```

To achieve good performance out of the DMAs, we generally aim to:

1. Move a large amount of contiguous data in each DMA buffer to amortize DMA buffer overhead
2. Move a large amount of data in each DMA transfer to amortize DMA transfer overhead.
3. Invoke as many parallel DMA transfers on the available DMA engines as possible.

These goals ultimately boil down to a quick optimization rule: maximize **both free (4KiB or above) and partition (ideally 128) dimension sizes** when moving tensors between SBUF and HBM using `nki.language.load` and `nki.language.store`. Refer to the [NKI Performance Guide](#) for more information on optimizing performance of data movements between HBM and SBUF.

Accessing SBUF/PSUM tensors using compute engines

Figure 58 shows a simplified timeline of how compute engines **stream** data in and out of on-chip SRAM (SBUF or PSUM). Refer to Figure 45 for the available connectivity between engines and SBUF/PSUM. At a high level, the compute engines are able to pipeline data reads, computation and writes along the F dimension of the src/dst tensors. In every cycle, each engine can read 128 elements across 128 SBUF/PSUM partitions, perform a computation on previously read 128 elements, and write 128 previously computed results to SBUF/PSUM. In other words, the P axis of a tensor is the *parallel* dimension for SBUF/PSUM data accessing, while the F axis of the tensor is the *time* dimension for data accessing.

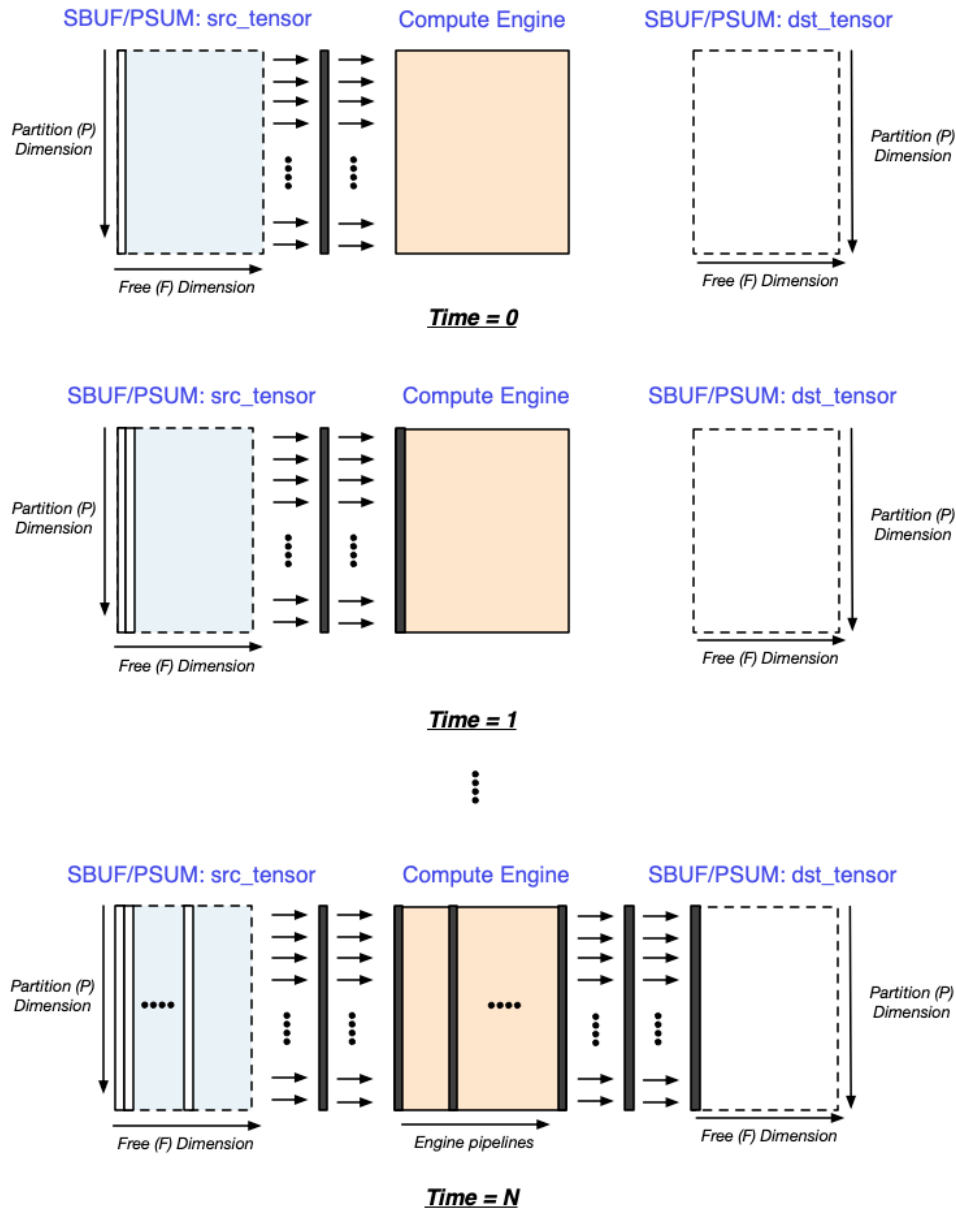


Fig. 58 Data streaming between SBUF and compute engine.

When accessing SBUF/PSUM tensors in an instruction, we need to follow different rules in the P and F dimensions. First, hardware does not allow P dimension striding when accessing data from a single SBUF/PSUM tensor. Therefore, a valid src/dst tensor of an instruction must occupy a continuous number of partitions. In addition, the hardware further enforces which partition a tensor can start from (`start_partition`) based on the number of partitions the tensor occupies (`num_partition`). This is currently handled by the tensor allocator in Neuron Compiler during NKI kernel compilation process:

- If `64 < num_partition <= 128`, `start_partition` must be 0
- If `32 < num_partition <= 64`, `start_partition` must be 0 or 64
- If `0 < num_partition <= 32`, `start_partition` must be one of 0/32/64/96

On the other hand, data accessing along the free dimension is a lot more flexible: the src/dst tensor of an engine instruction can support up to four-dimensional tensorized access pattern with a stride in each dimension within each partition. At the ISA level, each F axis in the tensor can have a size expressed in `uint16` and a stride expressed in `int16`, measured in data elements. As an example, if the tensor data type is BF16, the most-minor F dimension is set to 10, then we will stride across 20B within a partition at a time. [latest](#)

[Skip to main content](#)

[Indexing in NKI Programming Guide](#) to learn about how to index SBUF/PSUM tensors to achieve F dimension striding in NKI syntax.

Lastly, as implied in [Figure 58](#), when accessing a SBUF/PSUM tensor, all active partitions must follow the same F dimension access pattern. In other words, at every time step, the engine read/write interface will access data elements at the same *offset* within each active partition.

Cross-Partition Connectivity

The majority of VectorE/ScalarE/GpSimdE instructions on NeuronCore-v2 require `src_tensor` and `dst_tensor` to occupy the same number of partitions. When the number of partitions involved exceeds 64, by the `start_partition` rule discussed above, the `src_tensor` and `dst_tensor` in such cases must both start from partition 0. Therefore, we effectively cannot perform any cross-partition data movement when `num_partition > 64`: each partition of `src_tensor` data will eventually flow into the corresponding partition in `dst_tensor`.

However, when `num_partition < 64`, VectorE/ScalarE/GpSimdE on NeuronCore-v2 supports two styles of cross-partition SBUF/PSUM data movement patterns: 1) cross-half movement for `32 < num_partition <= 64` and 2) cross-quadrant movement for `0 < num_partition <= 32`. Figure below illustrates these two patterns for `num_partition=64` and `num_partition=32`. The shaded portion of the **Engine** block indicates the active lanes for the given instruction. With these movement patterns, each partition in `src_tensor` still has a one-to-one mapping to each partition in `dst_tensor`.

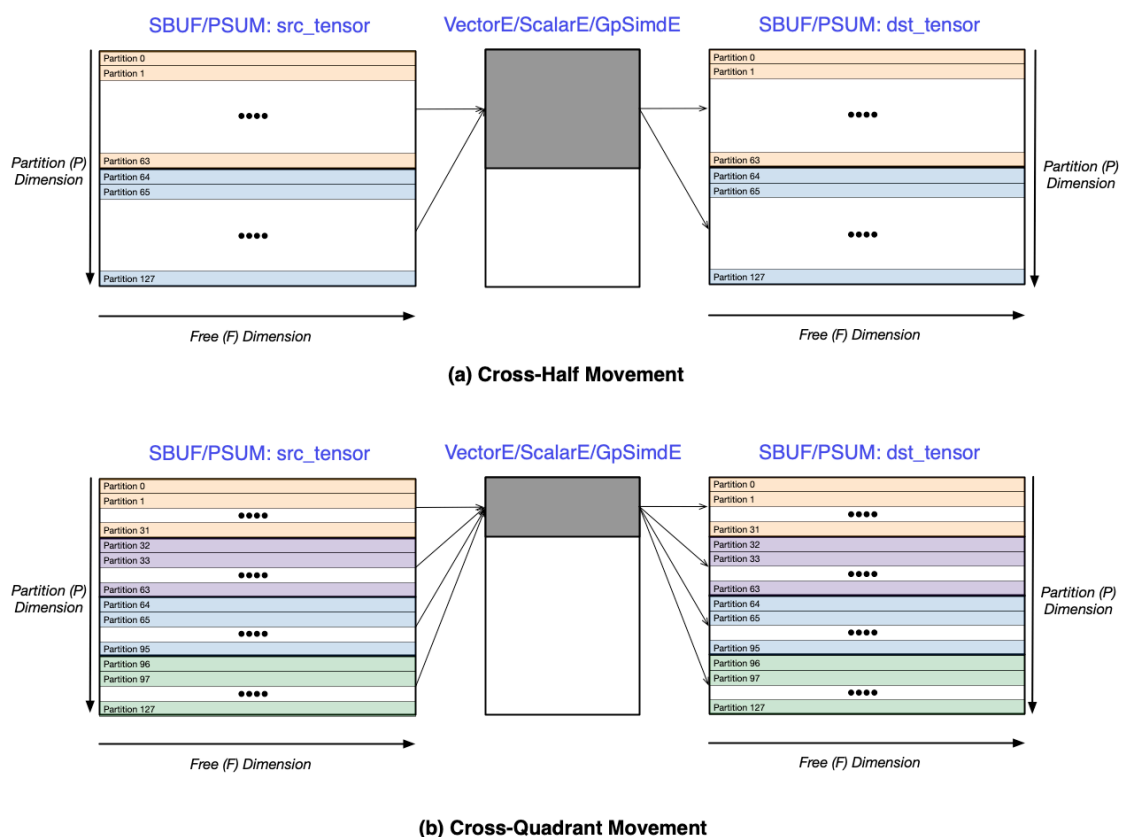


Fig. 59 Cross-partition connectivity.

Performance Consideration

Access pattern. As discussed previously in the context of compute engine utilization, it is recommended to use as

[Skip to main content](#)

bandwidth. In addition, accessing with a large stride in the most-minor (fastest) F dimension will incur performance penalty. When the most-minor F dimension stride is less than 16 bytes, SBUF/PSUM on NeuronCore-v2 can supply a peak bandwidth of 128 elements/cycle at 1.4 GHz for each tensor read/write interface. A 16-byte stride is equivalent to 4 elements for 32-bit data types, 8 elements for 16-bit data types or 16 elements for 8-bit data types. If the most-minor F dimension stride exceeds 16 bytes, the achievable bandwidth of each tensor read/write interface will be half of the peak bandwidth, which translates to roughly 50% performance hit on the instructions.

Concurrent SBUF/PSUM accesses by engines. As mentioned earlier, NeuronCore-v2 has the following on-chip RAM access restrictions:

1. Vector Engine and GpSimd Engine cannot access SBUF in parallel
2. Vector Engine and Scalar Engine cannot access PSUM in parallel

Despite these restrictions, SBUF is capable of driving peak bandwidth in each tensor read/write interface connected to VectorE/ScalarE/TensorE or GpSimdE/ScalarE/TensorE *simultaneously* without bandwidth interference. Similarly, PSUM can drive peak bandwidth for VectorE/TensorE or ScalarE/TensorE *simultaneously*.

Tensor access overhead. Initiating a tensor access request from an engine to its SBUF/PSUM read/write interface incurs a static overhead approximately 60 cycles on NeuronCore-v2. Compute engines can typically hide some of this latency through instruction level parallelism. However, it is still highly recommended to access tensors with large P and F dimension sizes whenever possible to amortize this overhead.

Near-memory accumulation in PSUM

As shown in [Figure 45](#), both VectorE and ScalarE have read and write access to PSUM, while TensorE only has write access. In fact, PSUM is designed to be a landing buffer for TensorE with near-memory accumulation capabilities that allows read-accumulate-write to every 4B element in memory. Note, this accumulation mechanism can *only* be controlled by TensorE. VectorE and ScalarE can only access PSUM like a regular SRAM similar to SBUF.

Next, let's discuss how TensorE can write outputs to PSUM. As previously discussed, PSUM is organized into 128 *partitions*, each consisting of 16KB of memory. Each partition is further divided into 8 PSUM banks, with each bank holding up to 512 32-bit values. The output tile of a TensorE matrix multiplication instruction (`nki.isa.nc_matmul`) must **fit** into one PSUM bank per partition, which is the fundamental reason for the **free dimension size limitation** for the `moving` tensor. Every `nc_matmul` instruction can choose whether to *override* existing bank data with instruction output or *accumulate* instruction output into existing bank data element-wise.

The accumulation mode of PSUM is particularly useful when the high-level matmul operator has a contraction dimension (i.e., `stationary/moving` partition dimension of `nki.isa.nc_matmul`) greater than 128. As an example, let's assume the following matmul dimensions:

- `x.shape = [128, 256]`
- `y.shape = [256, 512]`

Figure below shows this matmul mathematically and also how we would tile the contraction dimension. With tiling, we slice both `x` and `y` in the contraction dimension to get `[x0, x1]` and `[y0, y1]` input tiles. To get the final output result, we need to perform:

- `output0 = matmul(x0, y0)`

 [latest](#) ▼

[Skip to main content](#)

- $\text{output} = \text{output}_0 + \text{output}_1$

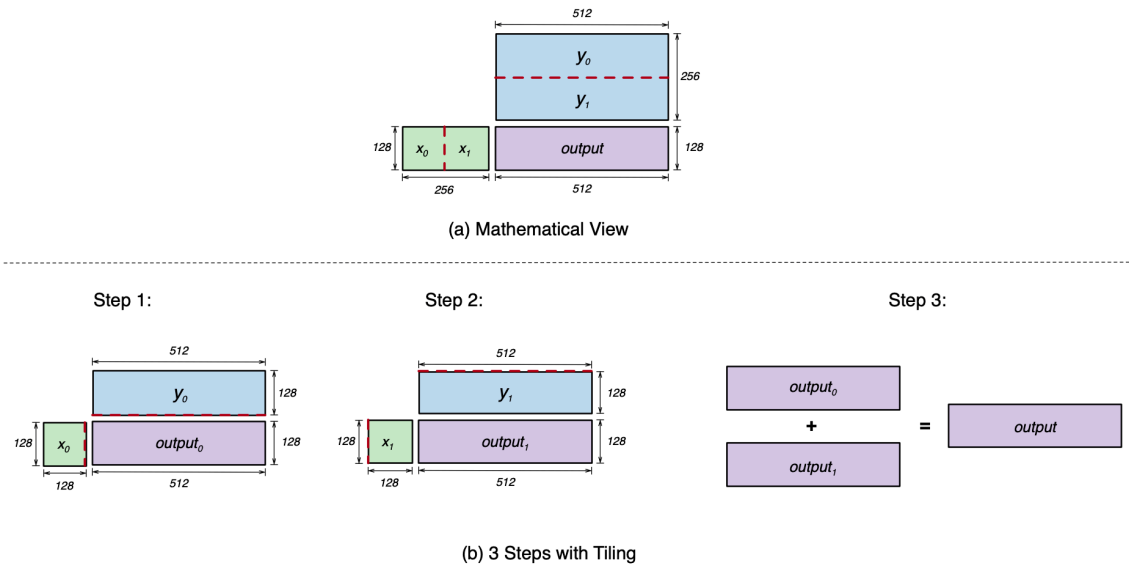


Fig. 60 Matmul tiling (mathematical view).

PSUM accumulation effectively combines Step 2 and 3 above into a single TensorE `nki.isa.nc_matmul` instruction. Assuming we have `x` in the transposed layout in SBUF, visually the above tiled matmul example will have two back-to-back `nki.isa.nc_matmul` instructions on TensorE:

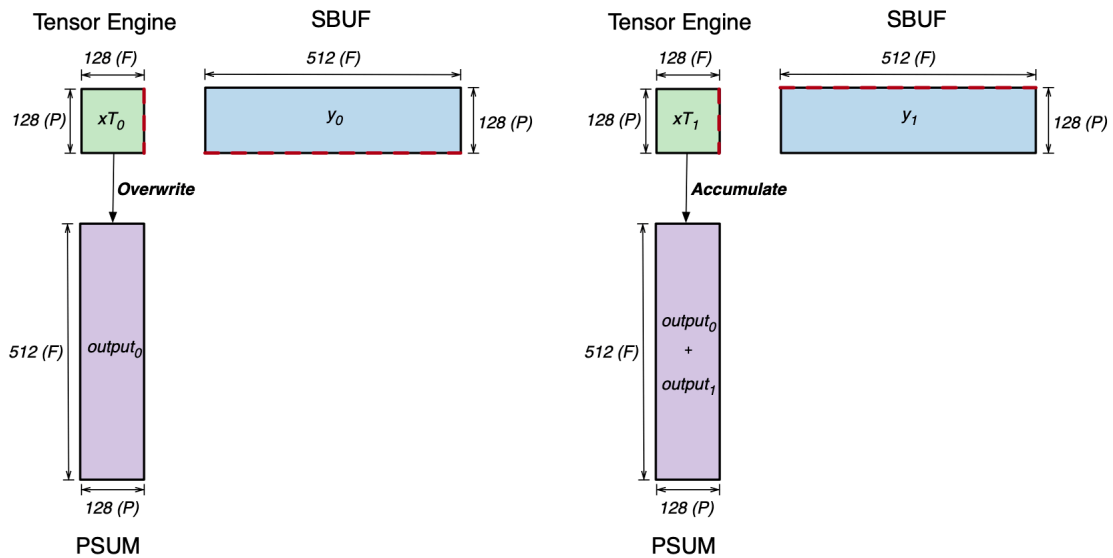


Fig. 61 Matmul tiling (hardware view).

Effectively, the first `nki.isa.nc_matmul` instruction overwrites the destination PSUM bank with the instruction output. The second instruction accumulates instruction output onto the previous instruction's result in the same PSUM. The PSUM accumulation is always done in FP32. A series of TensorE matmul instructions with the first one writing to a PSUM bank and more subsequent instructions accumulating into the same PSUM bank data is called a *matmul accumulation group*.

In NKI, `nisa.nc_matmul` supports an `accumulate` parameter to control PSUM accumulation behavior. When not specified (default), the compiler auto-detects: the first write to a PSUM bank overwrites, and subsequent writes accumulate. The following NKI code pattern demonstrates PSUM accumulation:

```
# condition 1: a psum buffer with zeros
psum_buf = nl.zeros((128, 128), dtype=nl.float32, buffer=nl.psum)

# condition 2: an affine range loop
for i in nl.affine_range(N):
    # condition 3: add matmul results from TensorEngine
    nisa.nc_matmul(dst=psum_buf, stationary=stationary_tile, moving=moving_tile)
```

Refer to the [Tiling Matrix Multiplications](#) tutorial for a detailed implementation.

Note

When `accumulate` is not specified (default), `nisa.nc_matmul` auto-detects accumulation: the first write to a PSUM location overwrites, and subsequent writes accumulate. Accumulation can also be controlled explicitly with `accumulate=True` or `accumulate=False`.

Finally, with 8 PSUM banks per partition, TensorE can have up to eight outstanding matmul accumulation groups, which allows flexible scheduling of matmul instructions on TensorE. Also, the extra buffering from multiple PSUM banks allows us to pipeline TensorE computation with other compute engines: TensorE can move onto the next accumulation group without waiting for VectorE/ScalarE to evict previous accumulation group results.

This document is relevant for: `Trn2`, `Trn3`